L27 - Array 1- One dimensional

Department of Computer Science
University of Pretoria

20 May 2024

Mathematically speaking:

An array is:

- An ordered list of homogenous (same data type) elements

-For notational purposes, each element of the array is subscripted by it's position in the array

$$A_1, A_2, A_3, \ldots, A_n$$

Computationally speaking:

An array is:

- a structure of related data items of the same data type

- C and C++ arrays are subscripted (indexed) from 0.

A[0], A[1], A[2], ..., A[n-1]

-Subscript (Index) operator is [ ] -Use when many related values need to be processed.

Array:

- variable that can store multiple values of the same type

- Values are stored in adjacent memory locations

- Declared using [ ] operator, e.g.: int tests[5]; - this declaration allocates the following memory:

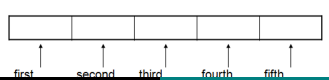
In the definition **int tests[5]**;

int - is the data type of the array elements

tests - is the name of the array

5, in [5] - is the size declarator. It specifies the number of elements in the array.

The size of an array is: (number of elements) * (size of each element) E.g. int tests[5] is an array of 20 bytes, assuming 4 bytes for an int



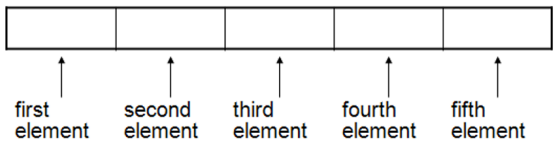
e.g. definition int tests[5];

-Named constants are commonly used as size declarators.

-const int SIZE = 5;

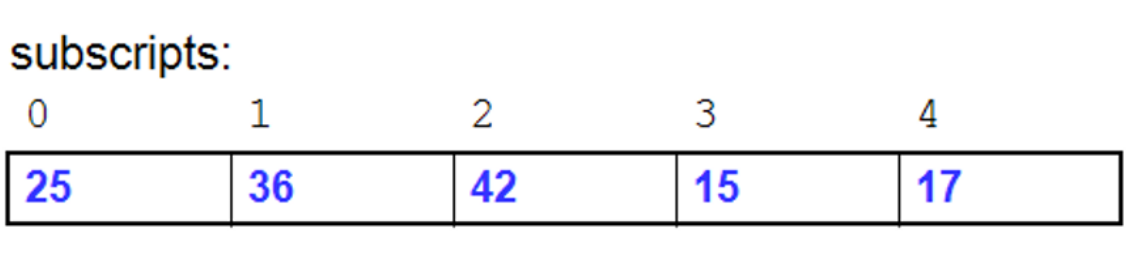
int tests[SIZE];

- The problem when the size of the array needs to be changed.



-Each element in an array is assigned a unique index (subscript).

-Indexes start at 0



The last element's index is n-1 where n is the number of elements in the array.

Array elements can be used as regular variables, e.g.:

tests[0] = 25;

cin >> tests[1];

Arrays must be accessed via individual elements

Program to read 5 test marks into an array and display the array contents

```
#include <iostream>
using namespace std;
int main() {
 const int SIZE = 5;
 int tests[SIZE];
    //read 5 marks values into the array
    cout << "Enter marks for 5 student:-\n";
    cout << "Student 1 mark:-";
    cin >> tests[0];
    cout << "Student 2 mark:-";
    cin >> tests[1];
    cout << "Student 3 mark:-";
    cin >> tests[2];
    cout << "Student 4 mark:-";
    cin >> tests[3];
    cout << "Student 5 mark:-";
    cin >> tests[4];
    return 0;}
```

```
int main()
{
    const int SIZE = 5;
    int tests[SIZE];
    //read 5 marks values into the array
    // as on last slide
   //display the 5 marks in the array
    cout << endl;
    cout << "Marks for the 5 student:" << endl;
    cout << "Student 1 mark:" << tests[0] << endl;
    cout << "Student 2 mark:" << tests[1] << endl;
    cout << "Student 3 mark:" << tests[2] << endl;
    cout << "Student 4 mark:" << tests[3] << endl;
    cout << "Student 5 mark:" << tests[4] << endl;
    return 0;}
```

QUESTION What if there are 100 students' marks to be processed?

ANSWER Use a for loop or a while loop or a do-while loop which executes 5 times.

Here is the general format of the range-based for loop:

for (dataType rangeVariable : array)
statement;

–Can access element with a constant value (literal) index (subscript):

```
cout << tests[3] << endl;
```

–Most commonly, we use an integer variable as an index (subscript)

```
int i = 3;
cout << tests[ i ] << endl;
```

E.g. Using a loop to step through an array:

```
const int SIZE = 5;
int tests[SIZE];

for (int index = 0; index < SIZE; index++)
cout << tests[index] << endl;
```

```
int main( )
{
    const int SIZE = 5;
    int index, studentNum, tests[SIZE];
    //read marks values into the array
    cout << "Enter marks for student:" << endl << endl;

    for (index = 0; index < SIZE; index++)
    {
        studentNum = index + 1;
        cout << "Mark for student " << studentNum << ":";
        cin >> tests[index];
    }
//display the marks in the array
cout << "Marks for the student:" << endl;
    for (index = 0; index < SIZE; index++)
    {
        studentNum = index + 1;
        cout << "Mark for student";
        cout << studentNum << ":";
        cout << tests[index] <<endl;
    }
```

—Arrays can be initialized with an initialization list:

```
const int SIZE = 5;
int values[SIZE]= {79,  82,  91,  77,  84};

const int MONTHS = 12;
string monthNames[MONTHS] ={"January",
      "February","March", "April","May","June",
  "July","August","September","October", "November","December"};
```

Values are stored in the array in the order in which they appear in the list.

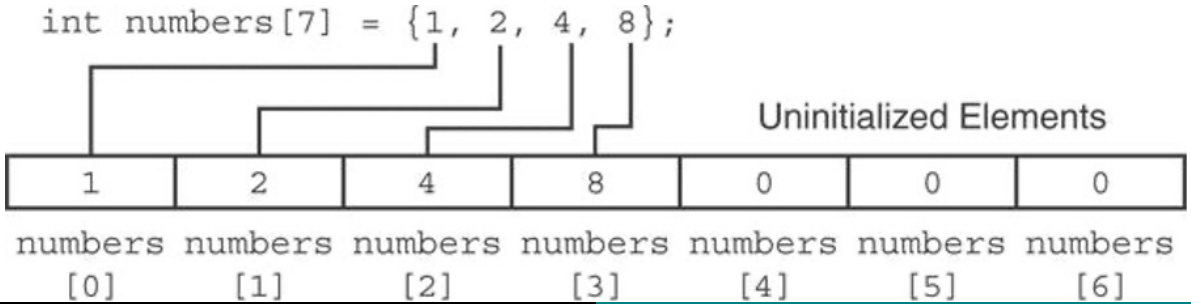
The initialization list size cannot exceed the array size

Partial initialization means that: Only some of the array elements are initialized

If an array is initialized with fewer initial values than the size declarator, the remaining elements will be set to 0

For example:

```
int numbers[ 7 ]    =    { 1, 2, 4, 8 };
```



```
1  // This program demonstrates the range-based for loop.
2  #include <iostream>
3  using namespace std;
4
5  int main()
6  {
7      // Define an array of integers.
8      int numbers[] = { 10, 20, 30, 40, 50 };
9
10     // Display the values in the array.
11     for (int val : numbers)
12         cout << val << endl;
13
14     return 0;
15 }
```

Figure: Example of range-based for loop

To define a function that takes an array parameter, use empty [ ] for the array:

```
//e.g. of function prototype:
void showMarks(int marks[ ]);
```

To pass an array as an argument to a function, use the array name:

```
int testMarks[SIZE];//array declaration
showMarks( testMarks ); //the function call
```

When defining a function which uses an array as a parameter:

-it is common to specify the array size so that function knows how many elements to process:

```
// e.g function prototype
void showMarks( int marks[ ], int size );
//alternatively
void showMarks( int [ ], int );
```

```
// function header
void showMarks(int marks[ ], int size)
```

The function call passes the array and its size to the function, e.g. in int main( ):

```
int testMarks[SIZE];
showMarks( testMarks, SIZE );
```

*** Why is showMarks(testMarks,SIZE) not called as showMarks(testMarks[ ],SIZE)???

Here are the reasons:

1.C++, when you pass an array to a function, you actually pass a pointer to the first element of the array

2.The function parameter int marks[] in showMarks is interpreted by the compiler as int* marks, which is a pointer to an integer

-This means the function can accept an array argument without specifying its size within the brackets.

3.When declaring a function that takes an array as an argument, you use the syntax int marks[] or int* marks. Both mean that the function takes a pointer to the first element of the array.

4.When you call the function, you just pass the name of the array (testMarks), which decays to a pointer to the first element.
5. testMarks in the function call showMarks(testMarks,SIZE) is sufficient because it represents the address of the first element of the array.

6. int marks[] or int* marks in the function definition tells the compiler that the function will receive a pointer to the first element of an integer array.
7. By passing testMarks to the function, you provide the function with the starting address of the array, which allows the function to access and iterate over the array elements using this pointer.

```
#include <iostream>
using namespace std;

// Function to print the elements of an array
void printArray(int arr[], int size) {
    cout << "Array elements:";
    for(int i = 0; i < size; i++) {
     cout << arr[i] << "-";
    }
    cout << endl;
}

int main() {
    // Define an array and specify its size
    int myArray[] = {10, 20, 30, 40, 50};
    int size = 5;  // Explicitly specify the size of the array

    // Call the function to print the array
    printArray(myArray, size);

    return 0;
}
```

e.g. int num = 149;
int *numPtr; //a pointer variable
numPtr = &num //store address of num in numPtr
(1) A pointer is an address e.g. 0x0010 (decimal 16)
(2) A pointer variable stores the memory address of a variable
e.g. numPtr stores the address of num

Two meanings of the * in the context of pointers

```
int    num = 149;
int    *numPtr;    //declare pointer variable
numPtr    =    &num;
cout << *numPtr; //dereferencing operator
```

Dereferencing **operator** *:
enables access to the value of the variable that the pointer points to.

In C++, the array name is a variable which stores the start address of the array in memory

E.g.

```
int values[ ] = { 4, 7, 11, 15, 19 };
cout << values; // displays start address of values
cout << values[ 0 ];   // displays 4

Array name can be used as a pointer constant:
cout << *values; // displays 4

Pointer can be used as an array name:
int *valPtr = values;
cout << valPtr[ 0 ];      // also displays 4
```

```
Given:
int values[ ] = { 4,  7,  11,  15,  19 };
int *valPtr = values;

What is valPtr + 1?
Answer:(address in valPtr ) + (1 * size of an int )
e.g. cout << *(valPtr + 1);//displays 7

 What is valPtr + 2?
 Answer:(address in valPtr ) + (2 *size of an int )
 e.g.cout << *(valPtr + 2);//displays 11
```

```cpp
#include <iostream>
using namespace std;
// Function to print the elements of an array using pointers
void printArray(int* arr, int size) {
    cout << "Array elements: ";
    for(int i = 0; i < size; i++) {
        cout << *(arr + i) << " "; // Using pointer arithmetic to
    }
    cout << endl;
}

int main() {
    // Define an array and specify its size
    int myArray[] = {10, 20, 30, 40, 50};
    int size = 5;  // Explicitly specify the size of the array

    // Call the function to print the array
    printArray(myArray, size);

    return 0;
}
```